

Physics Experiment Calculator

A command line Python project for five physics experiments

Project Report

Build Your Own Project (Evaluated Course Project)

Student name	ISHIKA CHOUDHARY
Registration number	26BAI10639
Course	PYTHON ESSENTIAL
Course code	CS1021
Platform	VITyarthi
GitHub repository	github.com/YOUR-USERNAME/YOUR-REPO-NAME

Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions and Rationale
8. Implementation Details
9. Screenshots and Results
10. Testing Approach
11. Challenges Faced
12. Learnings and Key Takeaways
13. Future Enhancements
14. References

1. Introduction

Physics practicals involve a lot of repeated calculation. For every experiment a student has to pick the right formula, put in the values, and keep track of units and signs. One small slip and the whole write up goes wrong. This project is a small calculator that takes care of that part.

The program is written in Python and runs completely in the terminal. It has a simple menu with five experiments: projectile motion, simple pendulum, radioactive decay, series LCR circuit and image formation by a concave lens. The user picks an experiment, types the values, and the program prints the results in a neat table. Every calculation is also saved in a log file.

The code is split into small modules so that each experiment lives in its own file, and the common work (input checking, printing and logging) is kept in a helpers package. The whole thing uses only the Python standard library.

2. Problem Statement

Students doing physics experiments waste time doing the same calculations by hand and often make mistakes with formulas, units and sign conventions. There is a need for a simple tool that gives correct results quickly and rejects wrong input instead of giving a wrong answer or crashing.

Objectives

- Build a menu driven command line program with five physics experiments.
- Give every experiment clear inputs and clearly labelled outputs.
- Check every input so wrong values cannot crash the program.
- Keep the code modular, with one module for each experiment.
- Keep a log of every calculation.
- Test the calculations with unit tests.

Scope and target users

The program only calculates from typed values. It has no GUI, no graphs and no sensor input. Target users are class 11 and 12 students, first year college students and teachers who want to check answers.

3. Functional Requirements

The project has five major functional modules, plus a menu and common helpers. The table shows the input and output of each module.

Module	Inputs	Outputs
Projectile motion	Initial speed (m/s), launch angle (degrees, 0 to 90)	Velocity in x direction, initial velocity in y direction, time of flight, maximum height, range
Simple pendulum	Length of string (m)	Time period, frequency
Radioactive decay	Starting amount, half life, time passed	Decay constant, mean life (tau), amount remaining, amount decayed
Series LCR circuit	L (H), C (F), R (ohm), source voltage (V), frequency (Hz)	XL, XC, impedance, current, phase angle, resonant frequency, quality factor

Module	Inputs	Outputs
Concave lens	Focal length, object distance, object height (cm)	Image distance, magnification, image height, nature of image

User workflow

- The program shows a menu with options 1 to 6.
- The user types a number to choose an experiment.
- The program asks for each value one by one and asks again if a value is wrong.
- The program calculates and prints the results in a table.
- The calculation is written to the log file.
- The menu is shown again until the user chooses 6 to exit.

4. Non-functional Requirements

Requirement	How the project meets it
Usability	Plain menu, prompts that state the unit and range, results printed in an aligned table with units.
Reliability	Every number typed is checked. Text, zero, negative values, nan and inf are rejected and the user is asked again, so bad input never crashes the program.
Error handling	Each calculate function raises a ValueError for impossible values. The main loop catches Ctrl+C and end of input and exits with a bye message.
Maintainability	One module per experiment, shared code in helpers, calculation code separate from input and output code.
Logging	Each calculation is saved with a timestamp, the inputs and the results in lab_log.txt.
Performance and resource use	All calculations are single formulas so results appear instantly. Only the standard library is used and nothing is kept in memory between runs.
Portability	Runs on Windows, Linux and Mac with Python 3.8 or newer and needs no installation of extra packages.

5. System Architecture

The program has three layers. The top layer is main.py, which shows the menu and calls the chosen experiment. The middle layer is the experiments package with one module for each experiment. The bottom layer is the helpers package, which has the input checker and the output and logging functions. The log file is the only storage used.

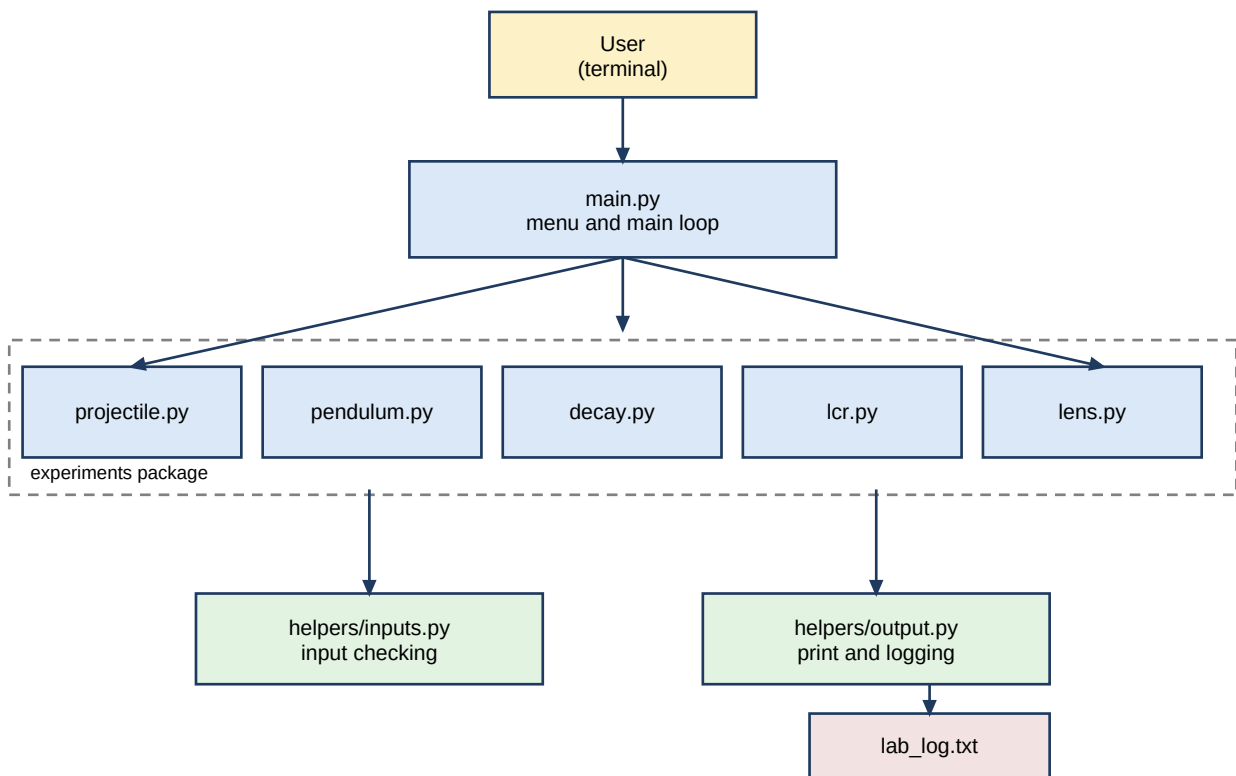


Figure 1: System architecture

6. Design Diagrams

6.1 Use case diagram

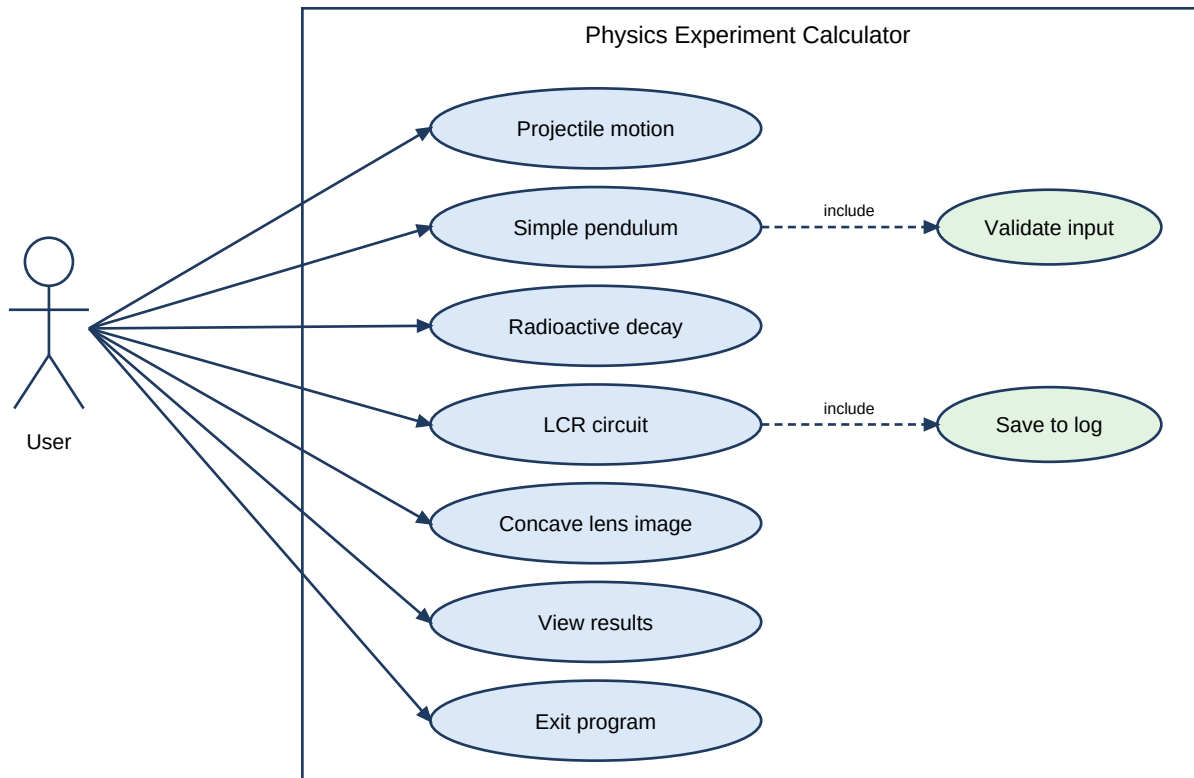


Figure 2: Use case diagram

6.2 Workflow diagram

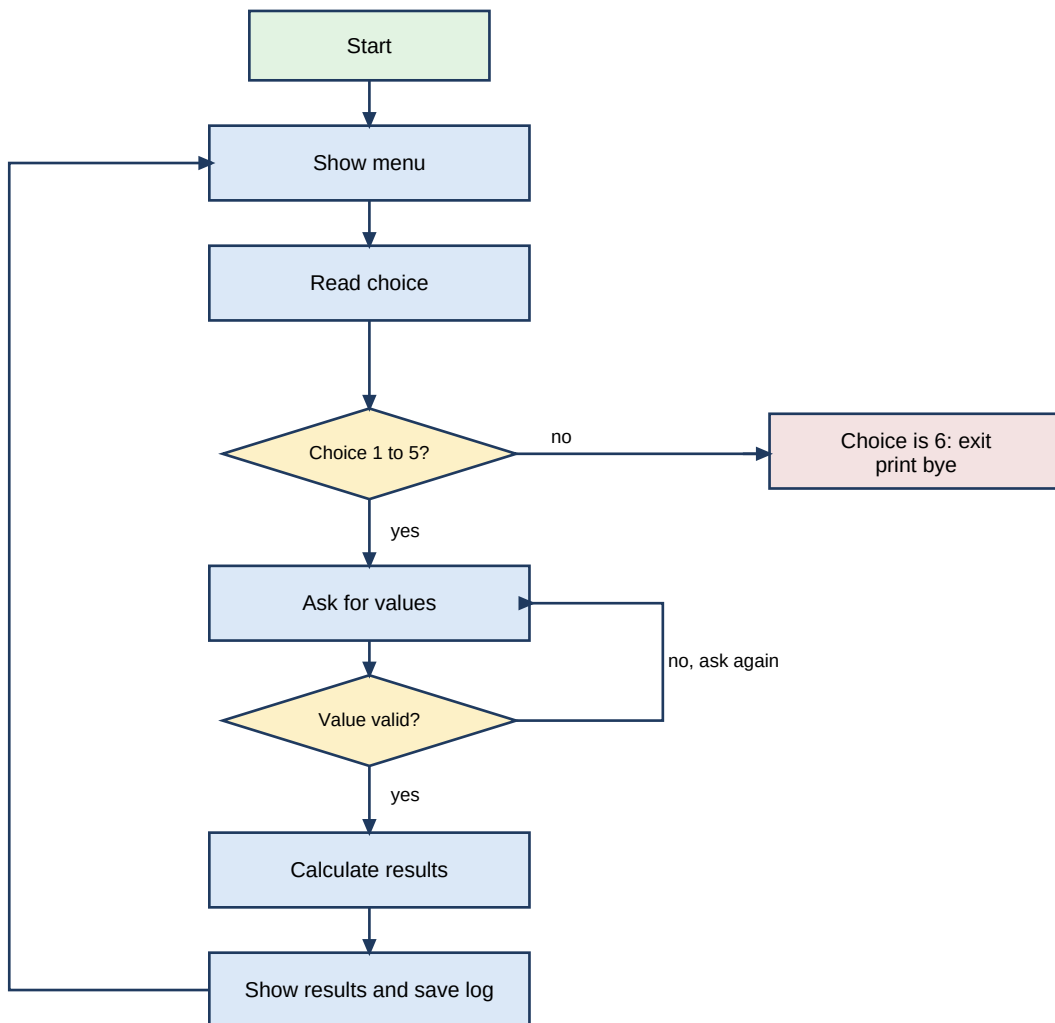


Figure 3: Workflow of the program

6.3 Sequence diagram

The sequence below is for one experiment run, from picking the option to coming back to the menu.

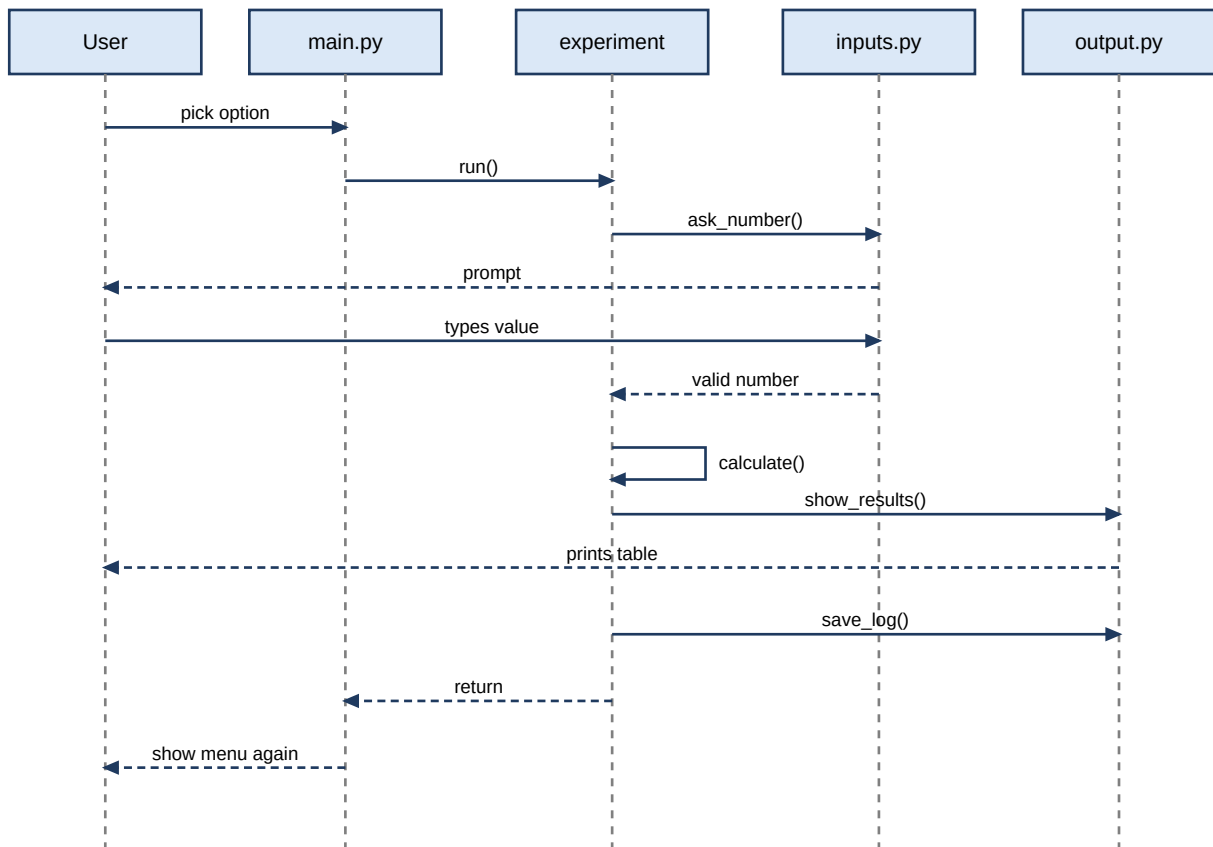


Figure 4: Sequence diagram for one calculation

6.4 Class and component diagram

The project uses functions in modules instead of classes, so the diagram shows components and the functions inside each one.

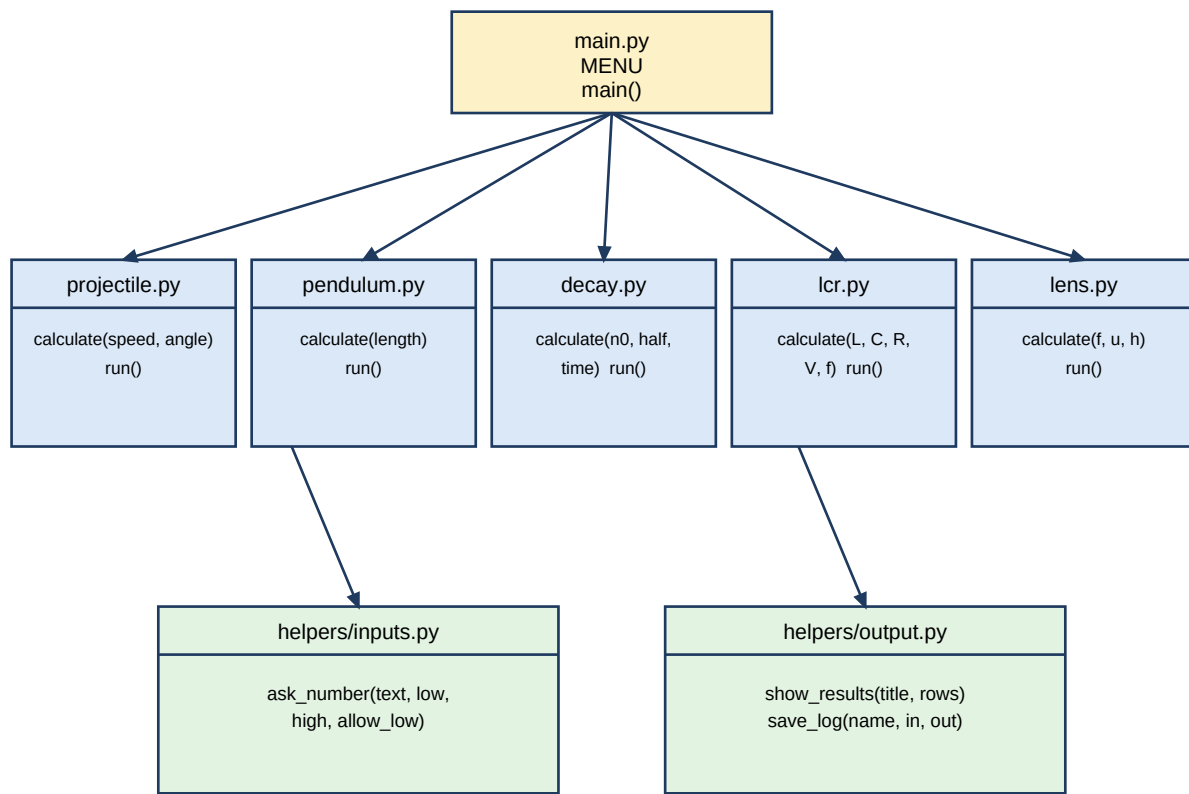


Figure 5: Component diagram

6.5 ER diagram

The project does not use a database, so an ER diagram is not needed. The only storage is a plain text log file where each line has the fields below.

Field	Example
Timestamp	2026-09-29 11:19:14,182
Experiment name	projectile
Inputs	{'speed': 25.0, 'angle': 45.0}
Results	{'vx': 17.67, 'vy': 17.67, ...}

7. Design Decisions and Rationale

- **Command line only.** The submission rules ask for a project that runs from a terminal, and it keeps the code simple.
- **Calculate and run are separate functions.** `calculate()` only does maths and returns a dictionary. `run()` handles typing and printing. This made unit testing easy because `calculate()` can be tested without typing anything.

- **One shared input function.** ask_number() is used everywhere so all experiments check input in the same way.
- **Half life as the decay input.** Half life is what students usually have in a problem, and tau is worked out from it as $\tau = \text{half life} / \ln 2$.
- **Sign handling in the lens module.** The user types positive distances and the program applies the Cartesian sign convention (f and u negative for a concave lens). This avoids sign mistakes.
- **Standard library only.** No installation is needed, so the evaluator can run it straight away.
- **Log file with delayed creation.** The file is only created when the first calculation is saved, so running the tests does not leave junk files.

8. Implementation Details

File	Purpose
main.py	Menu loop, calls the chosen experiment, handles Ctrl+C and end of input
helpers/inputs.py	ask_number() keeps asking until a valid number in range is typed
helpers/output.py	show_results() prints the table, save_log() writes to lab_log.txt
experiments/projectile.py	Projectile motion calculations
experiments/pendulum.py	Pendulum calculations
experiments/decay.py	Radioactive decay calculations
experiments/lcr.py	Series LCR circuit calculations
experiments/lens.py	Concave lens calculations
tests/	Unit tests for the experiments and the input function

Formulas used ($g = 9.8 \text{ m/s}^2$)

Experiment	Formulas
Projectile motion	$v_x = v \cos(a)$, $v_y = v \sin(a)$, time of flight $T = 2 v_y / g$, maximum height $H = v_y^2 / 2g$, range $R = v_x T$
Pendulum	$T = 2 \pi \sqrt{L / g}$, frequency = $1 / T$
Radioactive decay	$\lambda = \ln 2 / \text{half life}$, $\tau = 1 / \lambda$, $N = N_0 e^{-\lambda t}$
Series LCR	$X_L = 2 \pi f L$, $X_C = 1 / (2 \pi f C)$, $Z = \sqrt{R^2 + (X_L - X_C)^2}$, $I = V / Z$, phase = $\text{atan2}(X_L - X_C, R)$, $f_0 = 1 / (2 \pi \sqrt{LC})$, $Q = (1/R) \sqrt{L/C}$
Concave lens	$1/v - 1/u = 1/f$ with f and u negative, magnification $m = v / u$, image height = m x object height

The same pattern is used in every experiment module. First calculate() checks the values and raises a ValueError if something is impossible. Then it does the formulas and returns a dictionary. Then run() asks for the inputs with ask_number(), calls calculate(), builds a list of rows for show_results() and finally calls save_log(). Version control was done with Git and the work was committed step by step, starting with the helpers, then one commit for each experiment, then tests and documentation.

9. Screenshots and Results

The blocks below are real terminal output from the program. Values typed by the user are shown after each prompt.

Menu and a wrong option

```
=== Physics Experiment Calculator ===
```

1. Projectile motion
2. Simple pendulum
3. Radioactive decay
4. LCR circuit
5. Concave lens image
6. Exit

```
pick an option: 9
```

```
please pick a number from 1 to 6
```

1. Projectile motion
2. Simple pendulum
3. Radioactive decay
4. LCR circuit
5. Concave lens image
6. Exit

```
pick an option: 6
```

```
bye
```

Projectile motion with a text value and an out of range angle rejected

```
Projectile Motion
```

```
initial speed in m/s: abc
```

```
that is not a number, try again
```

```
initial speed in m/s: 25
```

```
launch angle in degrees (0 to 90): 100
```

```
value must be 90 or less
```

```
launch angle in degrees (0 to 90): 45
```

```
--- Projectile Motion Results ---
```

Velocity in x direction	17.6777 m/s
Initial velocity in y direction	17.6777 m/s
Time of flight	3.6077 s
Maximum height	15.9439 m
Horizontal range	63.7755 m

Pendulum with a negative length rejected

```
Simple Pendulum
```

```
this works best for small swing angles
```

```
length of the string in m: -2
```

```
value must be more than 0.0
```

```
length of the string in m: 1.5
```

```
--- Pendulum Results ---
```

Time period	2.4582 s
Frequency	0.4068 Hz

Radioactive decay: 1000 nuclei, half life 5, after 10 time units

```
Radioactive Decay
```

```
starting amount of nuclei (or mass): 1000
```

```
half life (any time unit): 5
```

```
time passed (same unit): 10
```

```
--- Radioactive Decay Results ---
```

Decay constant	0.1386 per time unit
Mean life (tau)	7.2135 time units
Amount remaining	250.0000
Amount decayed	750.0000

Series LCR circuit at 100 Hz

```
Series LCR Circuit
inductance L in henry: 0.1
capacitance C in farad (like 0.00001): 0.00001
resistance R in ohm: 50
source voltage in volt: 10
source frequency in Hz: 100

--- LCR Circuit Results ---
Inductive reactance          62.8319 ohm
Capacitive reactance         159.1549 ohm
Impedance                    108.5271 ohm
Current                      0.0921 A
Phase angle                  -62.5668 degrees
Resonant frequency          159.1549 Hz
Quality factor                2.0000
```

the circuit is more capacitive at this frequency

Concave lens image

```
Image by a Concave Lens
enter every distance as a positive number, the program handles signs
focal length in cm: 20
object distance in cm: 30
object height in cm: 4

--- Concave Lens Results ---
Image distance (v)           -12.0000 cm
Magnification                 0.4000
Image height                  1.6000 cm
```

the image is virtual, erect and smaller than the object
it forms on the same side of the lens as the object

Hand check of the results: for the projectile, $R = v^2 \sin(2\alpha) / g = 625 / 9.8 = 63.78$ m, which matches. For decay, 10 time units is two half lives so 1000 becomes 250, which matches. For the lens, $1/v = -1/20 - 1/30 = -1/12$, so $v = -12$ cm and $m = 0.4$, which matches.

10. Testing Approach

Testing is done with Python unittest. There are 20 tests in two files and all of them pass. Run them with: `python -m unittest discover -s tests -v`

Test group	What it checks
Projectile tests	45 degrees gives equal vx and vy and range v^2/g . Straight up gives the correct height and time. Bad speed and angle raise ValueError.
Pendulum tests	Period for 1 m is about 2.0071 s. Period times frequency is 1. Zero length raises ValueError.
Decay tests	After one half life half remains. tau equals half life / ln 2. Bad half life and negative time raise ValueError.
LCR tests	At resonance impedance equals R, phase is 0 and current is V / R . Q factor is correct. Zero resistance raises ValueError.
Lens tests	Image distance, magnification and image height match a hand calculation. Magnification is always between 0 and 1. Negative input raises ValueError.
Input tests	Text, blank, zero, negative, nan and inf are skipped. The upper limit and the allow_low option work.

Apart from the unit tests, the full program was run with scripted input to check the menu, wrong menu options, the exit option and the case where input ends suddenly. No crashes were found.

11. Challenges Faced

- **Sign convention for the lens.** The lens formula gives different answers depending on the sign convention used. It was solved by asking for positive distances and fixing the signs inside the code.
- **Input that looks like a number but is not.** Python accepts nan and inf as floats. These had to be rejected on purpose or the calculations would give meaningless output.
- **Testing code that asks for input.** Tests cannot type into a terminal. Keeping the maths in `calculate()` and using a fake input in the input tests solved this.
- **Wrong expected value in a test.** One test first failed because the expected pendulum period was taken for $g = 9.81$ while the code uses 9.8. The test value was corrected.
- **Floating point noise.** Values like $\cos 90$ degrees are not exactly zero. Results are printed to 4 decimal places and tests use `assertAlmostEqual`.

12. Learnings and Key Takeaways

- Keeping calculation code separate from input and output code makes programs much easier to test.
- Input checking is a big part of a reliable program, more than it looks at first.
- Working through the sign convention and the units of each formula gave a better understanding of the physics itself.
- Writing tests first for known cases, like resonance in the LCR circuit, catches formula mistakes early.
- Small commits with clear messages make the history of a project easy to follow.

13. Future Enhancements

- Add more experiments such as Ohm's law, Snell's law and Young's double slit.
- Add a parallel LCR mode and a convex lens mode.
- Let the user choose the value of g , for example for other planets.
- Save results to a CSV file so they can be opened in a spreadsheet.
- Draw graphs of the trajectory, the decay curve and the resonance curve with a plotting library.
- Add support for different unit systems.

14. References

- Python 3 documentation: math, logging and unittest modules, docs.python.org.
- Halliday, Resnick and Walker, Fundamentals of Physics.
- NCERT Physics textbooks for class 11 and class 12.
- Pro Git book by Scott Chacon and Ben Straub, git-scm.com.